

Rapport Technique : Prédiction de Trafic Urbain par Apprentissage de Graphe Contextuel (GNN)

Projet NYC Traffic Prediction (Vers Transfer Learning)

5 janvier 2026

Résumé

Ce document présente l'état d'avancement et la feuille de route pour le développement d'un modèle de prédiction de trafic basé sur la structure urbaine. L'objectif est d'entraîner un *Graph Neural Network* (GNN) capable de prédire la congestion sur un segment routier en fonction de ses caractéristiques topologiques (OpenStreetMap) et contextuelles (Points d'Intérêts, aménagements), sans dépendre de l'historique de trafic local. Cette approche vise une capacité de **généralisation inductive** (Zero-Shot Learning), permettant d'appliquer le modèle à de nouvelles villes (ex : Marseille) après un entraînement sur New York.

Table des matières

1	État des Lieux : Actifs et Données Disponibles	2
1.1	Structure du Graphe (Réseau Routier)	2
1.2	Données de Trafic (Vérité Terrain)	2
1.3	Outils de Traitement Existants	2
2	Approche Méthodologique : Graph Neural Networks (GNN)	2
2.1	Modélisation du Problème	2
3	Feuille de Route et Implémentation	3
3.1	Phase 1 : Enrichissement et Normalisation (Feature Engineering)	3
3.2	Phase 2 : Construction du Dataset (Edge-Centric)	3
3.3	Phase 3 : Architecture du Modèle	3
3.4	Phase 4 : Entraînement et Validation Spatiale	3
4	Conclusion et Prochaines Étapes	3

1 État des Lieux : Actifs et Données Disponibles

Nous disposons actuellement d'un jeu de données complet modélisant le réseau routier de New York et l'historique de trafic associé sur la période 2010-2013.

1.1 Structure du Graphe (Réseau Routier)

Le réseau est modélisé sous forme de graphe orienté $G = (V, E)$ statique provenant d'OpenStreetMap.

- **Nœuds** (V) : Définis dans `nodes.csv`. Chaque nœud représente une intersection avec un identifiant unique et des coordonnées géographiques.
- **Arêtes** (E) : Définies dans `links.csv`. Chaque lien connecte deux nœuds. **Point d'attention** : Pour assurer la transférabilité vers d'autres villes (ex : France), nous ne pouvons pas nous fier uniquement à la classification administrative américaine.
 - `street_length` : Longueur du segment en mètres.
 - `osm_class` : Type de route.
 - **Attributs Physiques (Estimés)** : Nombre de voies (`lanes`) et largeur approximative, cruciaux pour distinguer une avenue américaine d'une rue européenne.
 - Géométrie : Angles de début et de fin.

1.2 Données de Trafic (Vérité Terrain)

Les estimations de trafic sont dérivées de trajets de taxis.

- **Granularité** : Une estimation par heure.
- **Métriques** : `travel_time` (Target principale) et `num_trips` (Utilisé pour masquer les données manquantes lors de l'entraînement).

1.3 Outils de Traitement Existants

Un script de pré-traitement (`preprocess.py`) a été développé pour :

1. Charger le graphe en mémoire.
2. Calculer la vitesse moyenne pondérée.
3. Classifier la congestion en 4 niveaux.

2 Approche Méthodologique : Graph Neural Networks (GNN)

L'hypothèse centrale est que la congestion est une fonction de la **topologie** et du **contexte**. Nous utilisons un GNN inductif pour apprendre la corrélation spatiale universelle.

2.1 Modélisation du Problème

Contrairement aux GNN classiques qui prédisent sur les nœuds, notre cible Y est sur les arêtes. Soit un graphe où chaque arête e_{ij} possède un vecteur de caractéristiques X_{ij} . Nous cherchons à apprendre f :

$$Y_{ij}^{(t)} = f(G, X_{ij}, C_{ij}, T)$$

Où C_{ij} représente les données contextuelles **normalisées** (indépendantes de l'échelle de la ville).

3 Feuille de Route et Implémentation

3.1 Phase 1 : Enrichissement et Normalisation (Feature Engineering)

Pour garantir que le modèle fonctionne sur Marseille après entraînement sur NYC, les features ne doivent pas être absolues.

Action : Créer `enrich_graph.py` générant des attributs relatifs :

- **Densité Commerciale (Z-Score)** : Densité de commerces normalisée par rapport à la moyenne de la ville (et non un compte brut).
- **Centralité Relative** : Percentile de distance au centre (0 = centre, 1 = périphérie extrême), plutôt qu'une distance en km.
- **Contexte Temporel** : Encodage cyclique de l'heure (Sin/Cos) pour capturer la "nature" de l'heure plutôt que sa valeur.

3.2 Phase 2 : Construction du Dataset (Edge-Centric)

Le format standard `torch_geometric` est orienté nœuds. Nous devons adapter la structure.

Stratégie "Dual Graph" (Line Graph) : Pour prédire sur les routes, nous transformons le graphe :

- Les **Routes** deviennent des **Nœuds** du GNN.
- Les **Intersections** deviennent les **Liens** entre ces nœuds.

Ceci permet d'utiliser des architectures GNN classiques (SAGE, GAT) directement sur les objets qui nous intéressent (les segments de route).

3.3 Phase 3 : Architecture du Modèle

Architecture proposée (Sur le Dual Graph) :

1. **Input Layer** : Encodage des features de route (Largeur, POIs normalisés, Type).
2. **Message Passing Layers** (x3) : Agrégation de l'état des routes adjacentes. Le modèle apprend : "Comment une route se comporte si ses voisines sont de type résidentiel avec forte densité commerciale?".
3. **Readout Layer** : MLP (Multi-Layer Perceptron) final pour la régression de la vitesse.

3.4 Phase 4 : Entraînement et Validation Spatiale

- **Loss Function** : MSE sur les segments ayant des données (`num_trips > 0`).

— **Validation Inductive** :

- **Train** : Quartiers de Brooklyn/Manhattan.
- **Test** : Quartier du Queens (jamais vu).

Si la validation sur le Queens est concluante, le transfert vers une nouvelle carte (Marseille) sera techniquement valide.

4 Conclusion et Prochaines Étapes

Le pivot vers une architecture "Dual Graph" et la normalisation des données contextuelles sont les clés pour réussir le transfert vers une autre ville sans historique.

Prochaine action : Développement du script de conversion du graphe routier en *Line Graph* (Graphe Dual).